

1. Features of ES6 (ECMAScript 2015)

ES6 introduced significant enhancements to JavaScript, including:

New Syntax Features:

- Arrow Functions - Shorter function syntax with lexical `this` binding
- Template Literals - String interpolation using backticks (```)
- Destructuring - Extract values from arrays/objects into variables
- Default Parameters - Default values for function parameters
- Rest/Spread Operators (`...`) - Collect or spread elements
- Enhanced Object Literals - Shorthand properties and methods

New Data Types & Structures:

- `let` and `const` - Block-scoped variable declarations
- Symbol - Unique primitive values
- Map, Set, WeakMap, WeakSet - New collection types

Object-Oriented Features:

- Classes - Syntactic sugar over prototype-based inheritance
- Class Inheritance - `extends` and `super` keywords
- Getters/Setters - Property access control

Asynchronous Programming:

- Promises - Native promise implementation
- Generators - Functions that can be paused and resumed
- Async/Await (ES7 but related) - Simplified promise handling

Module System:

- ES Modules - `import/export` statements

Other Features:

- `for...of` loops - Iterate over iterable objects
 - Iterators/Iterables - Custom iteration protocols
 - `Array.from()`, `Array.of()` - New array methods
 - `Object.assign()` - Object merging
 - `Number.isInteger()`, `Number.isNaN()` - Improved number methods
-

2. JavaScript `let`

The `let` keyword declares block-scoped variables that can be reassigned.

Key Characteristics:

- Block Scope - Variable exists only within the block `{ }` where declared
- No Hoisting (Temporal Dead Zone) - Cannot access before declaration
- Can be Reassigned - Values can be changed after declaration
- Cannot be Redeclared in the same scope

Syntax:

```
javascript
```

```
let variableName = value;
```

Examples:

```
javascript
```

```
// Block scope  
if (true) {
```

```

    let x = 10;
    console.log(x); // 10
}
console.log(x); // ReferenceError: x is not defined

// Reassignment
let count = 5;
count = 10; // ✅ Valid

// Temporal Dead Zone
console.log(a); // ReferenceError
let a = 5;

// Cannot redeclare
let b = 1;

let b = 2; // SyntaxError: Identifier 'b' has already been declared

```

3. Differences Between var and let

Feature	var	let
Scope	Function-scoped	Block-scoped ({})
Hoisting	Hoisted and initialized with <code>undefined</code>	Hoisted but not initialized (Temporal Dead Zone)
Redeclaration	Allowed in same scope	Not allowed in same scope
Reassignment	Allowed	Allowed
Global Object Property	Attaches to <code>window</code> (in browsers)	Does not attach to <code>window</code>

Best Practice

Avoid in modern JavaScript

Preferred for mutable
variables

Examples:

javascript

```
// var: Function-scoped
function testVar() {
  if (true) {
    var x = 5;
  }
  console.log(x); // 5 (accessible outside block)
}

// let: Block-scoped
function testLet() {
  if (true) {
    let y = 5;
  }
  console.log(y); // ReferenceError (not accessible)
}

// var: Hoisting
console.log(z); // undefined (hoisted)
var z = 5;

// let: Temporal Dead Zone
console.log(w); // ReferenceError
let w = 5;

// var: Can redeclare
var a = 1;
var a = 2; // ✅ Valid

// let: Cannot redeclare
let b = 1;

let b = 2; // ❌ SyntaxError
```

4. JavaScript `const`

The `const` keyword declares block-scoped constants that cannot be reassigned.

Key Characteristics:

- Block Scope - Same as `let`
- Cannot be Reassigned - Value cannot change after initialization
- Must be Initialized - Declaration and assignment must be together
- No Hoisting - Subject to Temporal Dead Zone
- Mutable Objects - Object properties can still be modified

Syntax:

```
javascript
```

```
const CONSTANT_NAME = value;
```

Examples:

```
javascript
```

```
// Constant assignment
```

```
const PI = 3.14159;
```

```
PI = 3.14; // ❌ TypeError: Assignment to constant variable
```

```
// Must initialize
```

```
const x; // ❌ SyntaxError: Missing initializer
```

```
// Block scope
```

```
if (true) {
```

```
    const max = 100;
```

```
}
```

```
console.log(max); // ReferenceError
```

```
// Object mutation (allowed)
const user = { name: 'John' };
user.name = 'Jane'; //  Valid (modifying property)
user = { name: 'Jane' }; //  Invalid (reassigning object)

// Array mutation (allowed)
const numbers = [1, 2, 3];
numbers.push(4); //  Valid

numbers = [1, 2, 3, 4]; //  Invalid (reassigning array)
```

5. ES6 Class Fundamentals

ES6 classes provide a cleaner syntax for creating objects and handling inheritance.

Basic Class Structure:

```
javascript

class ClassName {
  // Constructor method (called when using `new`)
  constructor(parameters) {
    // Initialize properties
    this.property = value;
  }

  // Instance methods
  methodName() {
    // Method logic
  }

  // Static methods (called on class, not instances)
  static staticMethod() {
    // Static method logic
  }

  // Getters and setters
  get property() {
```

```
        return this._property;
    }

    set property(value) {
        this._property = value;
    }
}
```

Example:

javascript

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    greet() {
        return `Hello, my name is ${this.name}`;
    }

    static compareAge(person1, person2) {
        return person1.age - person2.age;
    }
}

// Creating instances
const john = new Person('John', 25);
console.log(john.greet()); // "Hello, my name is John"

console.log(Person.compareAge(john, new Person('Jane', 30))); // -5
```

Key Features:

- Constructor - Called when instantiating with `new`
- Methods - Functions defined inside the class body
- Static Methods - Belong to the class, not instances
- Getters/Setters - Computed properties with access control

- `super` - Reference to parent class (in inheritance)
-

6. ES6 Class Inheritance

Inheritance allows a class to extend another class, inheriting its properties and methods.

Syntax:

javascript

```
class ChildClass extends ParentClass {
  constructor(params) {
    super(params); // Call parent constructor
    // Additional child initialization
  }

  // Override parent method
  methodName() {
    super.methodName(); // Call parent method
    // Additional logic
  }
}
```

Example:

javascript

```
// Parent class
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    return `${this.name} makes a sound.`;
  }
}
```



```

    }

    move() {
        return `${this.name} moves.`;
    }
}

// Child class
class Dog extends Animal {
    constructor(name, breed) {
        super(name); // Call parent constructor
        this.breed = breed;
    }

    // Override speak method
    speak() {
        return `${this.name} barks!`;
    }

    // Additional method
    fetch() {
        return `${this.name} fetches the ball.`;
    }
}

// Usage
const rex = new Dog('Rex', 'German Shepherd');
console.log(rex.speak()); // "Rex barks!" (overridden)
console.log(rex.move()); // "Rex moves." (inherited)

console.log(rex.fetch()); // "Rex fetches the ball." (new method)

```

Inheritance Features:

- `extends` - Specifies the parent class
- `super()` - Calls parent constructor (must be called before `this`)
- Method Overriding - Child can redefine parent methods
- Method Extending - Child can add new methods
- Property Access - Child instances have access to parent properties
- `super.method()` - Call parent method from child

7. ES6 Arrow Functions

Arrow functions provide a shorter syntax and lexically bind the `this` value.

Syntax:

```
javascript

// Basic syntax
(param1, param2) => expression

// Single parameter (parentheses optional)
param => expression

// No parameters
() => expression

// Block body (requires explicit return)
(param1, param2) => {
  // statements
  return value;
}

// Returning an object (wrap in parentheses)
() => ({ key: 'value' })
```

Examples:

```
javascript

// Traditional function
const add = function(a, b) {
  return a + b;
};

// Arrow function
```

```

const addArrow = (a, b) => a + b;

// Single parameter
const double = x => x * 2;

// No parameters
const greet = () => 'Hello';

// Block body
const multiply = (a, b) => {
    const result = a * b;
    return result;
};

// With array methods
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(n => n * 2); // [2, 4, 6, 8]

const evens = numbers.filter(n => n % 2 === 0); // [2, 4]

```

Lexical **this** Binding:

javascript

```

const obj = {
    name: 'Alice',
    // Traditional function - this depends on caller
    greetTraditional: function() {
        setTimeout(function() {
            console.log(this.name); // undefined (this = window/global)
        }, 1000);
    },
    // Arrow function - this lexically binds
    greetArrow: function() {
        setTimeout(() => {
            console.log(this.name); // 'Alice' (this = obj)
        }, 1000);
    }
};

```

Key Differences from Traditional Functions:

- No `this` binding - Uses lexical `this` from surrounding scope
 - Cannot be used as constructors - No `new` keyword
 - No `arguments` object - Use rest parameters instead
 - Cannot be used as generators - No `yield` keyword
 - Shorter syntax - More concise for simple operations
-

8. `set()` and `map()`

Set

A Set is a collection of unique values where each value can occur only once.

Key Features:

- Stores unique values (automatically removes duplicates)
- Values can be of any type (primitives or objects)
- Maintains insertion order
- Iterable (can use `for...of` loops)
- Efficient for membership testing

Syntax:

```
javascript

// Create a Set
const set = new Set();
const set2 = new Set([1, 2, 3, 3, 4]); // [1, 2, 3, 4] (duplicates removed)

// Methods
set.add(value);           // Add a value
set.delete(value);        // Remove a value
set.has(value);           // Check if value exists
set.size;                 // Get number of elements
set.clear();              // Remove all elements
```

```
// Iteration
for (const value of set) { /* ... */ }

set.forEach(value => { /* ... */ });
```

Examples:

javascript

```
// Creating and using Set
const uniqueNumbers = new Set([1, 2, 3, 3, 4, 4, 5]);
console.log(uniqueNumbers); // Set(5) {1, 2, 3, 4, 5}
console.log(uniqueNumbers.size); // 5

// Adding values
uniqueNumbers.add(6);
uniqueNumbers.add(3); // Ignored (already exists)

// Checking existence
console.log(uniqueNumbers.has(3)); // true
console.log(uniqueNumbers.has(10)); // false

// Removing values
uniqueNumbers.delete(2); // true
uniqueNumbers.delete(10); // false

// Iteration
uniqueNumbers.forEach(value => console.log(value));
// Output: 1, 3, 4, 5, 6

// Converting Set to Array
const array = [...uniqueNumbers]; // [1, 3, 4, 5, 6]
const array2 = Array.from(uniqueNumbers); // [1, 3, 4, 5, 6]

// Practical use: Remove duplicates from array
const numbers = [1, 2, 2, 3, 4, 4, 5];

const unique = [...new Set(numbers)]; // [1, 2, 3, 4, 5]
```

Map

A Map is a collection of key-value pairs where keys can be any data type.

Key Features:

- Keys can be any type (objects, functions, primitives)
- Maintains insertion order
- Iterable (can use `for...of` loops)
- Better performance for frequent additions/removals
- Direct access to size

Syntax:

```
javascript

// Create a Map
const map = new Map();
const map2 = new Map([['key1', 'value1'], ['key2', 'value2']]);

// Methods
map.set(key, value);    // Add or update entry
map.get(key);           // Get value by key
map.has(key);           // Check if key exists
map.delete(key);        // Remove entry by key
map.size;               // Get number of entries
map.clear();            // Remove all entries

// Iteration
for (const [key, value] of map) { /* ... */ }

map.forEach((value, key) => { /* ... */ });
```

Examples:

```
javascript

// Creating and using Map
const userMap = new Map();

// Setting values
```

```
userMap.set('name', 'John');
userMap.set('age', 25);
userMap.set({id: 1}, 'User 1');

// Getting values
console.log(userMap.get('name')); // "John"
console.log(userMap.get('age')); // 25

// Checking existence
console.log(userMap.has('name')); // true
console.log(userMap.has('email')); // false

// Size
console.log(userMap.size); // 3

// Using objects as keys
const key1 = {name: 'key1'};
const key2 = {name: 'key2'};
const objectMap = new Map();
objectMap.set(key1, 'Value for key1');
objectMap.set(key2, 'Value for key2');
console.log(objectMap.get(key1)); // "Value for key1"

// Iteration
objectMap.forEach((value, key) => {
  console.log(`${JSON.stringify(key)}: ${value}`);
});

// Converting Map to Array
const entries = Array.from(objectMap); // [[key1, value1], [key2, value2]]
const keys = Array.from(objectMap.keys()); // [key1, key2]
const values = Array.from(objectMap.values()); // [value1, value2]

// Practical use: Counting occurrences
const words = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple'];
const wordCount = new Map();
words.forEach(word => {
  wordCount.set(word, (wordCount.get(word) || 0) + 1);
});

console.log(wordCount); // Map(3) {"apple" => 3, "banana" => 2, "orange" => 1}
```

Set vs Map Comparison

Feature	Set	Map
Purpose	Store unique values	Store key-value pairs
Key Type	Values are keys	Any type (including objects)
Access	By value	By key
Duplicate Values	Not allowed	Allowed (different keys)
Iteration Order	Insertion order	Insertion order
Methods	<code>add, has, delete</code>	<code>set, get, has, delete</code>
Use Case	Uniqueness, membership	Key-value mapping, caching

WeakSet and WeakMap (Bonus)

WeakSet:

- Holds only objects (no primitives)
- References are weak (garbage collected)
- No iteration methods
- Use for detecting circular references

WeakMap:

- Keys must be objects (no primitives)
- References are weak (garbage collected)
- No iteration methods
- Use for private data or caching

javascript


```
// WeakSet example  
const weakSet = new WeakSet();  
const obj = {data: 'value'};  
weakSet.add(obj);  
console.log(weakSet.has(obj)); // true  
// If obj is garbage collected, weakSet entry is removed
```

```
// WeakMap example  
const weakMap = new WeakMap();  
const key = {};  
weakMap.set(key, 'private data');  
console.log(weakMap.get(key)); // 'private data'  
  
// If key is garbage collected, weakMap entry is removed
```